Finding Time Complexity - Counter Method

Convert each algorithm into a program and find its time complexity using the counter method.

Question 1

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i = 1;
    int s = 1;
    while (s <= n)
    {
        i++;
        s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

For example:

Input	Result
9	12

Answer (program)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int i = 1;
    int s = 1;
    int count = 2;

    while (1)
    {
        count++;

        if (s > n)
            break;

        count++;
        i++;

        count++;
        s += i;
    }

    printf("%d", count);

    return 0;
}
```

Sample run

Input: **9** Output (counter value): **12**

Time complexity

s takes the values 1, 3, 6, 10, ... i.e. s = i(i+1)/2. The loop stops when s > n, so i is about sqrt(2n). The loop body runs about sqrt(2n) times with a constant number of counter increments each time.

Time complexity: O(sqrt(n))

Question 2

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("*");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

int count=0;
void func(int n)
{
    if(n==1)
    {
        count++;
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            count++;
            for(int j=1; j<=n; j++)
            {
                count++;
                count++;
                count++;

                break;
            }
            count++;
        }
        count++;
    }
    count++;
    printf("%d",count);
}
int main(){
    int n;

    scanf("%d",&n);

    func(n);

    return 0;
}
```

Sample run

Input: **5** Output (counter value): **27**

Time complexity

The inner loop executes only once per outer iteration because of the break statement. So the outer loop runs n times and each iteration does a constant amount of work.

Time complexity: O(n)

Question 3

Convert the following algorithm into a program and find its time complexity using the counter method.

```
Factor(num) {
{
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

void Factor(int num)
{
    int count = 0;
    for (int i = 1; i <= num; ++i)
    {
        count++;
        count++;

        if (num % i == 0)
        {
            count++;
        }
    }
    count++;
    printf("%d", count);
}

int main()
{
    int n;
    scanf("%d",&n);

    Factor(n);

    return 0;
}
```

Sample run

Input: **6** Output (counter value): **17**

Time complexity

The loop runs from 1 to num, so it executes num times. Inside it there is one modulus check (and an occasional print) which costs constant time.

Time complexity: O(n)

Question 4

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function(int n)
{
    int c = 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++){
        c++;

        for(int j=1; j<n; j = 2 * j){
            c++;

            for(int k=1; k<n; k = k * 2){
                c++;
                c++;

            }
            c++;
        }
        c++;
    }
    c++;
    c++;

    printf("%d",c);
}
int main()
{
    int n;
    scanf("%d",&n);

    function(n);

    return 0;
}
```

Sample run

Input: **8** Output (counter value): **106**

Time complexity

The outer loop (i from n/2 to n) runs n/2 times. The j loop doubles each time, so it runs log n times. The k loop also doubles each time, so it runs log n times. Total work = (n/2) x log n x log n.

Time complexity: $O(n \log^2 n)$

Question 5

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void reverse(int n)
{
    int rev = 0, remainder;
    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
    }
    print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

void reverse(int n)
{
    int rev = 0, remainder;
    int c=0;
    while (n != 0) {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;
        c+=4;
    }
    c+=3;
    printf("%d",c);

}

int main()
{
    int n;
    scanf("%d",&n);

    reverse(n);

    return 0;
}
```

Sample run

Input: **1234** Output (counter value): **19**

Time complexity

Each iteration removes one digit of n (n is divided by 10). A number n has about log10(n) digits, so the loop runs log10(n) times with constant work per iteration.

Time complexity: O(log n)